Task 3 - Prototype Implementation: MongoDB + AirBnB Abhinav Bachu, Hanshul Bahl, Alejandra Arias CS 498 / Data Management in the Cloud 04/20/2026

1. Stage 3 Goal and What Changed Since Stage 2

Stage 2 was the design: four collections, eight compound indexes, and pseudocode for all six queries from the AirBnB question document. Stage 3 turns that design into a working prototype that boots a MongoDB instance, loads real InsideAirbnb data for four cities, and runs three of the six queries end-to-end with `explain('executionStats')` capture so we can show actual index usage rather than promised index usage.

The data model and indexes are unchanged from Stage 2 (collections: `listings`, `calendar`, `reviews`, `neighborhoods`; indexes verbatim from §2.2 of the Stage 2 report). Everything else - the loader, the query implementations, the driver, the evidence capture - is new in Stage 3. The prototype lives in `stage3-prototype/` and runs with a single `npm start`.

2. Implemented Subset

The assignment requires "(some of) the queries from Part 1" and at least two queries demonstrated. We picked three queries chosen specifically to cover the three distinct query *styles* in the design:

#	Query	Style	Why this one
Q1	Portland 2-day search	filter + group + lookup + sort	Cleanest demonstration of the `calendar {city, date, available}` compound index and the `listings {city, listing_id}` lookup index working together.
Q5	December reviews per city per year	pure aggregation with date extraction	Shows the date-as-string design choice paying off (no `$toDate` conversion needed for month/year extraction).
Q6	Re-book reminder + same-host listings	multi-step orchestration across all 4 collections	The most complex query in the set; touches every collection and validates that the multi-stage approach from Stage 2 actually works.

We deliberately skipped Q3 and Q4 (contiguous-availability windows and per-month bookable-night totals) because both rely on the same per-listing run-segmentation logic and that logic is awkward to express purely in MongoDB aggregation - the Stage 2 report already noted this and proposed a hybrid application-side pass. Building it for the prototype would have added one large chunk of code that demos exactly the same MongoDB capabilities Q1 already demos. Q2 (neighborhoods with no listings in a target month) is essentially a set difference layered on top of the same `calendar` index pattern as Q1, so it would have added breadth but not depth.

3. Loader Architecture and Data Volume Loaded

3.1 Architecture

The loader (`src/a.js`) is a single function `load(db)` that:

1. Iterates the four cities (Los Angeles, Portland, Salem, San Diego).
2. For each city, fetches `listings.csv.gz`, `reviews.csv.gz`, `calendar.csv.gz`, and `neighbourhoods.csv` directly from `data.insideairbnb.com`.
3. Streams each gzipped CSV through `zlib.createGunzip()` into `csv-parse` so we never hold a full file in memory.
4. Caps the per-city row count via the `L` table (500 listings, 1000 reviews, 200 neighborhoods, 2000 calendar rows). The cap matches the planned proof-of-concept scale: enough to make all three queries return non-empty results, small enough that the whole pipeline runs in under 10 seconds end-to-end on a laptop.
5. Cleans rows: numeric coercion with currency-symbol stripping, `t/f` to boolean for availability, description truncation to 200 chars, and dedup of neighborhoods on `(city, neighborhood)` to satisfy the unique index.
6. Bulk-inserts each collection.
7. Creates the eight indexes from Stage 2 §2.2 verbatim.

The driver (`src/e.js`) wraps the loader in a `mongodb-memory-server` instance so the prototype is fully self-contained: no external Mongo install, no Atlas account, no docker. The same code would run unmodified against a real cluster by passing a different `MongoClient` URI.

3.2 Volumes loaded in the recorded run

```
listings:       1780
reviews:        4000
neighborhoods:  411
calendar:       8000
```

Source: `out/load_evidence.txt` (committed to the repo). The same file contains sample documents from each collection and a printout of all indexes so the loaded state is fully auditable without re-running the prototype.

4. Query Implementations and Execution Plans

Each query is one source file that exports `run(db, opts)` and `explain(db, opts)`. The driver calls both, writes results to `out/qN_results.json` and the full `executionStats` to `out/explain_qN.json`. Numbers below come from those files.

4.1 Q1 - Portland 2-day search

```
[
  { $match: { city: "portland", date: { $in: dates }, available: true } },
  { $group: { _id: "$listing_id", a: { $sum: 1 } } },
  { $match: { a: dates.length } },
```

```
  { $lookup: {
      from: "listings",
      let: { l: "$_id" },
      pipeline: [
        { $match: { $expr: { $and: [
          { $eq: ["$city", "portland"] },
          { $eq: ["$listing_id", "$$l"] }
        ]}}},
        { $project: { _id: 0, name: 1, neighborhood: 1, room_type: 1,
                      accommodates: 1, property_type: 1, amenities: 1,
                      price: 1, review_scores_rating: 1 } }
      ],
      as: "b"
  }},
  { $unwind: "$b" }, { $replaceRoot: { newRoot: "$b" } },
  { $sort: { review_scores_rating: -1, listing_id: 1 } },
  { $limit: 25 }
]
```

The driver picks the demo dates dynamically by querying the loaded calendar for the earliest pair of consecutive available dates in Portland, so the query always has live data to operate on. In the recorded run the picked window was `2025-12-04` to `2025-12-05` and the query returned 3 listings, top result "*Beautiful Apt near City Center!*" (rating 4.94).

Execution plan summary (from `out/explain_q1.json`):

- `$match` stage uses **IXSCAN on** `city_1_date_1_available_1` with bounds `["portland","portland"]`, `["2025-12-04","2025-12-04"]`, `["2025-12-05","2025-12-05"]`, `[true,true]`. No collection scan.
- `nReturned: 3`, `totalKeysExamined: 6`, `totalDocsExamined: 6`, `executionTimeMillis: 3`. Tight - one key and one doc per matching `(listing, date)` pair.
- `$lookup` against `listings` uses **IXSCAN on** `city_1_listing_id_1` (the unique index), `nReturned: 3`, `totalDocsExamined: 3`.
- `$sort` operates on 3 docs, no spill, no disk usage.

This is exactly the plan the Stage 2 design was aiming for.

4.2 Q5 - December reviews per city per year

```
[
  { $match: { date: { $regex: /^\d{4}-12-\d{2}$/ } } },
  { $group: {
      _id: { city: "$city", year: { $substr: ["$date", 0, 4] } },
      review_count: { $sum: 1 }
```

```
  }},
  { $project: { _id: 0, city: "$_id.city", year: "$_id.year", review_count: 1
} },
  { $sort: { city: 1, year: 1 } }
]
```

Returned 53 (city, year) groups in the recorded run. Sample rows include `los_angeles` 2024 and 2023 having the highest December review counts of any city/year pair in the loaded subset.

Execution plan note: this is a full-collection aggregation by design - we want every December review counted, so there is no useful index prefix to apply unless we add `{date: 1}` as a leading single-field index. The Stage 2 design intentionally did not include that index because the only date-driven `reviews` query in the question document is Q5 itself, and Q5 already runs in 2 ms over 4000 rows. If the dataset grew to tens of millions of reviews, adding `reviews { date: 1 }` would be the first tuning step.

4.3 Q6 - Re-book reminder + same-host listings

Q6 is the multi-step orchestration query. Source: `src/d.js`.

Step 1 (one Mongo aggregation) - find every (city, listing, reviewer) triple that has more than one review:

```
[
  { $match: { reviewer_id: { $ne: null } } },
  { $group: {
      _id: { city: "$city", listing_id: "$listing_id", reviewer_id:
"$reviewer_id" },
      review_dates: { $push: "$date" },
      review_count: { $sum: 1 }
  }},
  { $match: { review_count: { $gt: 1 } } }
]
```

Step 2 (per repeat-review group, in application code) - for each prior review date:

- compute the `[YYYY-MM-01, YYYY-MM-31]` month bounds;
- one `findOne` against `calendar` with `{city, listing_id, date in [bounds], available: true}` -> `available_in_same_month`;
- one aggregate against `calendar` with the same filter to get min/max nights for the month;
- one `findOne` against `listings` for description, host, URL;
- one `find` against `listings` for other listings by the same host in the same city (`{city, host_id, listing_id: {$ne: this}}`).

The driver caps output at 25 records to keep the JSON file readable; the `limit` parameter is configurable.

Recorded run returned 25 enriched records, including repeat reviews for "*RUN Runyon, Beau Furn Rms Terrace Hollyw Hill View*" (host *Chas.*) showing one other listing under the same host in Los Angeles.

Execution plan note: Step 1's explain (in `out/explain_q6.json`) shows a `$group` over the full `reviews` collection - this is intentional, the query needs to inspect every reviewer-listing pair. Step 2's per-record reads all hit indexes:

- `calendar` reads use `city_1_listing_id_1_date_1` (the second compound index from Stage 2, optimized exactly for this access pattern);
- `listings` reads use either `city_1_listing_id_1` (unique) or `host_id_1_city_1` (for the same-host enumeration);
- `reviews` doesn't need to be re-read after Step 1.

The fact that we never had to add a new index for Q6 is itself a validation of the Stage 2 design: every per-record read in the multi-step pipeline lands on an index that was defined for Q1, Q3, or Q6's stated needs.

5. Critique - What We'd Change in the Data Model

Now that we've actually built and exercised the queries, two things stand out:

1. **Dates as strings was the right call but for an unexpected reason.** Stage 2 reasoned about strings vs `Date` mostly for storage efficiency. In practice, what mattered was that *Q5's date-extraction stays simple*: `{ $substr: ["$date", 0, 4] }` is one operator and indexable as a regex prefix; the same query with `Date` requires `{ $year: "$date" }` and forces a full-collection compute regardless of indexes. If we were redoing Stage 2, we would phrase this trade-off in terms of query simplicity, not just storage.

2. **The `host_id_1_city_1` index has the wrong key order.** Stage 2 picked it for Q6's "other listings by same host in same city" lookup. In actual use the lookup *also* filters by `listing_id != currentListing`, and the cardinality is dominated by `host_id` (most hosts have 1-2 listings). Reversing it to `{city: 1, host_id: 1}` would let us share a prefix with `{city: 1, listing_id: 1}` and `{city: 1, neighborhood: 1, room_type: 1}`, which all start with `city`. The current order works, it's just slightly off-pattern.

A third smaller item: `neighborhoods` is essentially a join lookup table. If we had Q2 in the prototype we'd be hitting it constantly. We'd consider promoting the canonical neighborhood list into a per-city array embedded into a single small "city metadata" document - it's read-only data and small enough that a one-document-per-city shape would let us skip the join entirely.

6. Lessons Learned

- **`mongodb-memory-server` is fantastic for design iteration.** The total feedback loop from "edit a query" to "see explain output" is under 10 seconds because we never wait on container or

cluster startup. We cannot recommend it enough for prototyping; it would have shaved real time off Stage 2 too.

- **`explain('executionStats')` is the single most valuable Mongo tool for design validation.** Pseudocode in Stage 2 was guessing about index usage. One line of `aggregate(...).explain('executionStats')` per query in Stage 3 turned guessing into proof.
- **Multi-step orchestration in application code is fine and often clearer.** Q6 could have been forced into a single aggregation with `$lookup`s nested inside `$lookup`s. The version we built is straight-line JS that calls four single-purpose helpers. It is much easier to read, debug, and explain in a presentation, and the per-record cost is tiny because each helper hits an index.
- **Schema design happens at index design time, not at collection design time.** All three of our queries' performance characteristics were determined by the compound index choices, not by what was in the documents. We spent a lot of Stage 2 time on document shapes that turned out not to matter much; we should have spent more on access patterns earlier.

7. Advice for Future Teams

- Pick one query first and design every collection and every index around it. Then add the second query and only modify what's necessary. Doing the entire schema in one pass tempts you to add indexes "just in case" - we did this in Stage 2 with `host_id_1_city_1` and ended up with the wrong key order.
- Use `mongodb-memory-server` from day one. Do not start by setting up Atlas.
- Always commit `explain('executionStats')` output alongside query results. It's the only way reviewers (and your future self) can tell whether the query is running the way you think it is.
- Resist the temptation to denormalize early. We considered embedding `calendar` arrays inside `listings` in Stage 2 and decided against it; in Stage 3 the per-listing calendar would have been hundreds of array elements and updates would have been painful.

8. References

1. MongoDB Documentation. https://www.mongodb.com/docs/manual/
2. MongoDB Aggregation Pipeline. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
3. MongoDB Indexes. https://www.mongodb.com/docs/manual/indexes/
4. MongoDB `explain()` and `executionStats`. https://www.mongodb.com/docs/manual/reference/method/cursor.explain/
5. `mongodb-memory-server`. https://github.com/typegoose/mongodb-memory-server
6. `csv-parse`. https://csv.js.org/parse/
7. InsideAirbnb Data Portal. https://insideairbnb.com/get-the-data.html
8. AirBnB Assignment Query Document. `Twitter-info-questions.pdf` / `AirBnB-info-questions.pdf`
9. Stage 2 Report. `task3-deliverables/TASK_3_Report_MongoDB_Airbnb.pdf`